

```
# =====  
# SCENARIO 1: AI-Powered Drone for Emergency Medicine Delivery  
# Greedy Best-First Search (GBFS) and A* Search  
# =====
```

```
class Graph:
```

```
    """A simple weighted graph used to represent the drone's road network."""
```

```
    def __init__(self):
```

```
        self.edges = {}    # node -> list of (neighbor, cost)
```

```
        self.heuristics = {} # node -> estimated straight-line distance to goal
```

```
    def add_edge(self, a, b, cost):
```

```
        self.edges.setdefault(a, []).append((b, cost))
```

```
        self.edges.setdefault(b, []).append((a, cost)) # undirected roads
```

```
    def set_heuristic(self, node, value):
```

```
        self.heuristics[node] = value
```

```
    def neighbors(self, node):
```

```
        return self.edges.get(node, [])
```

```
def greedy_best_first_search(graph, start, goal):
```

```
    """
```

GBFS: $f(n) = h(n)$

Chooses the node that LOOKS closest to the goal, ignoring cost already spent.

Returns (path, total_actual_cost).

"""

```
frontier = [(graph.heuristics[start], start)]
```

```
came_from = {start: None}
```

```
visited = set()
```

```
while frontier:
```

```
    _, current = heapq.heappop(frontier)
```

```
    if current == goal:
```

```
        return _reconstruct_path(came_from, goal), _path_cost(graph, came_from, goal)
```

```
    if current in visited:
```

```
        continue
```

```
    visited.add(current)
```

```
    for neighbor, cost in graph.neighbors(current):
```

```
        if neighbor not in visited:
```

```
            came_from.setdefault(neighbor, current)
```

```
            heapq.heappush(frontier, (graph.heuristics.get(neighbor, math.inf), neighbor))
```

```
return None, math.inf
```

```

def a_star_search(graph, start, goal):
    """
    A*:  $f(n) = g(n) + h(n)$ 
    Balances actual travel cost  $g(n)$  with estimated remaining distance  $h(n)$ .
    Returns (path, total_actual_cost).
    """
    frontier = [(graph.heuristics[start], 0, start)] # (f, g, node)
    g_score = {start: 0}
    came_from = {start: None}
    visited = set()

    while frontier:
        f, g, current = heapq.heappop(frontier)

        if current == goal:
            return _reconstruct_path(came_from, goal), g

        if current in visited:
            continue

        visited.add(current)

        for neighbor, cost in graph.neighbors(current):
            tentative_g = g + cost

            if neighbor not in g_score or tentative_g < g_score[neighbor]:
                g_score[neighbor] = tentative_g
                came_from[neighbor] = current

```

```
f_score = tentative_g + graph.heuristics.get(neighbor, math.inf)
heapq.heappush(frontier, (f_score, tentative_g, neighbor))
```

```
return None, math.inf
```

```
def _reconstruct_path(came_from, goal):
    path = [goal]
    while came_from[path[-1]] is not None:
        path.append(came_from[path[-1]])
    path.reverse()
    return path
```

```
def _path_cost(graph, came_from, goal):
    path = _reconstruct_path(came_from, goal)
    total = 0
    for a, b in zip(path, path[1:]):
        for neighbor, cost in graph.neighbors(a):
            if neighbor == b:
                total += cost
                break
    return total
```

```
def demo_scenario_1():
```

```

print("\n=== SCENARIO 1: Drone Delivery (GBFS vs A*) ===")

g = Graph()

# Recreating the "Route A (flooded, short heuristic)" vs
# "Route B (safe, higher heuristic)" example from the document.

g.add_edge("Start", "A", 5) # looks close (h=5) but flooded -> costly in reality
g.add_edge("Start", "B", 7) # looks farther (h=7) but safe/cheap
g.add_edge("A", "Hospital", 10) # flooded road -> expensive actual cost
g.add_edge("B", "Hospital", 2) # safe road -> cheap actual cost


g.set_heuristic("Start", 8)
g.set_heuristic("A", 5)      # small heuristic -> GBFS is tempted
g.set_heuristic("B", 7)
g.set_heuristic("Hospital", 0)


gbfs_path, gbfs_cost = greedy_best_first_search(g, "Start", "Hospital")
astar_path, astar_cost = a_star_search(g, "Start", "Hospital")


print(f"GBFS path: {gbfs_path} | actual cost: {gbfs_cost}")
print(f"A* path: {astar_path} | actual cost: {astar_cost}")
print("-> A* avoids the flooded shortcut because it accounts for real travel cost.")


# =====
# SCENARIO 2: Smart City Traffic Signal Optimization
# Hill Climbing, Simulated Annealing, Genetic Algorithm
# =====

```

```
def traffic_cost(signal_plan):
    """
    A toy objective function to MINIMIZE.

    signal_plan: list of green-light durations (seconds) for each intersection.

    Ideal duration per intersection is arbitrarily set at 30s;
    cost penalizes deviation from ideal AND unbalanced timing between neighbors.
    """

    ideal = 30

    cost = sum((t - ideal) ** 2 for t in signal_plan)

    # Penalize big jumps between neighboring intersections (poor coordination)
    for a, b in zip(signal_plan, signal_plan[1:]):
        cost += 0.5 * (a - b) ** 2

    return cost
```

```
def random_neighbor(plan, step=5, low=10, high=60):
    """Generate a neighboring plan by nudging one random intersection's timing."""

    new_plan = plan[:]

    i = random.randrange(len(plan))

    delta = random.choice([-step, step])

    new_plan[i] = max(low, min(high, new_plan[i] + delta))

    return new_plan
```

```
def hill_climbing(initial_plan, max_iterations=500):
```

```
"""
```

Hill Climbing: always move to a better neighbor; stop at first local optimum.

Can get stuck in local maxima/plateaus/ridges (see document section ii).

```
"""
```

```
current = initial_plan[:]
```

```
current_cost = traffic_cost(current)
```

```
for _ in range(max_iterations):
```

```
    neighbor = random_neighbor(current)
```

```
    neighbor_cost = traffic_cost(neighbor)
```

```
    if neighbor_cost < current_cost:
```

```
        current, current_cost = neighbor, neighbor_cost
```

```
    # else: reject (this is what causes it to get stuck on local optima)
```

```
return current, current_cost
```

```
def simulated_annealing(initial_plan, max_iterations=2000,
```

```
    start_temp=100.0, cooling_rate=0.995):
```

```
"""
```

Simulated Annealing: sometimes accepts WORSE solutions (probability

depends on temperature) to escape local optima. Temperature cools over time.

```
"""
```

```
current = initial_plan[:]
```

```
current_cost = traffic_cost(current)
```

```
best, best_cost = current[:], current_cost
```

```
temperature = start_temp
```

```
for _ in range(max_iterations):
```

```
    neighbor = random_neighbor(current)
```

```
    neighbor_cost = traffic_cost(neighbor)
```

```
    delta = neighbor_cost - current_cost
```

```
    if delta < 0 or random.random() < math.exp(-delta / max(temperature, 1e-9)):
```

```
        current, current_cost = neighbor, neighbor_cost
```

```
        if current_cost < best_cost:
```

```
            best, best_cost = current[:], current_cost
```

```
    temperature *= cooling_rate
```

```
return best, best_cost
```

```
def genetic_algorithm(num_intersections, population_size=30, generations=100,
```

```
    mutation_rate=0.1, low=10, high=60):
```

```
    """
```

```
    Genetic Algorithm: evolves a population of candidate signal plans
```

```
    (chromosomes) using selection, crossover, and mutation.
```

```
    """
```

```
def random_individual():
```

```
    return [random.randint(low, high) for _ in range(num_intersections)]
```



```
def fitness(individual):  
    # Lower traffic_cost = higher fitness  
    return 1.0 / (1.0 + traffic_cost(individual))
```

```
def select_parent(population, fitnesses):  
    # Roulette-wheel selection  
    total = sum(fitnesses)  
    pick = random.uniform(0, total)  
    running = 0  
    for ind, fit in zip(population, fitnesses):  
        running += fit  
        if running >= pick:  
            return ind  
    return population[-1]
```

```
def crossover(parent1, parent2):  
    point = random.randint(1, num_intersections - 1)  
    return parent1[:point] + parent2[point:]
```

```
def mutate(individual):  
    return [  
        max(low, min(high, gene + random.choice([-5, 5]))) if random.random() <  
mutation_rate  
        else gene  
        for gene in individual  
    ]
```

```
population = [random_individual() for _ in range(population_size)]
```

```
best, best_cost = None, math.inf
```

```
for _ in range(generations):
```

```
    fitnesses = [fitness(ind) for ind in population]
```

```
    for ind, cost in zip(population, [traffic_cost(i) for i in population]):
```

```
        if cost < best_cost:
```

```
            best, best_cost = ind[:], cost
```

```
    new_population = []
```

```
    while len(new_population) < population_size:
```

```
        p1 = select_parent(population, fitnesses)
```

```
        p2 = select_parent(population, fitnesses)
```

```
        child = mutate(crossover(p1, p2))
```

```
        new_population.append(child)
```

```
    population = new_population
```

```
return best, best_cost
```

```
def demo_scenario_2():
```

```
    print("\n=== SCENARIO 2: Traffic Signal Optimization ===")
```

```
    random.seed(42)
```

```

num_intersections = 5

initial_plan = [random.randint(10, 60) for _ in range(num_intersections)]

print(f"Initial plan: {initial_plan} | cost: {traffic_cost(initial_plan):.2f}")


hc_plan, hc_cost = hill_climbing(initial_plan)

print(f"Hill Climbing    -> plan: {hc_plan} | cost: {hc_cost:.2f}")


sa_plan, sa_cost = simulated_annealing(initial_plan)

print(f"Simulated Annealing -> plan: {sa_plan} | cost: {sa_cost:.2f}")


ga_plan, ga_cost = genetic_algorithm(num_intersections)

print(f"Genetic Algorithm  -> plan: {ga_plan} | cost: {ga_cost:.2f}")

print("-> GA typically finds the lowest-cost (best) plan for large search spaces.")

```

```

# =====

```

```

# SCENARIO 3: Autonomous Mars Rover - Online Search Agent

```

```

# =====

```

```

class MarsGrid:

```

```

    """

```

```

    The TRUE environment (hidden from the rover). '#' = obstacle, '.' = free.

```

```

    The rover only discovers cells as it gets close to them (partial observability).

```

```

    """

```

```

    def __init__(self, grid):

```

```
self.grid = grid

self.rows = len(grid)

self.cols = len(grid[0])
```

```
def is_free(self, pos):

    r, c = pos

    if 0 <= r < self.rows and 0 <= c < self.cols:

        return self.grid[r][c] != '#'

    return False
```

```
def online_dfs_agent(true_env, start, goal, sensor_range=1):
```

```
    """
```

A simple Online Search Agent (online DFS-style) for the rover.

It does NOT know the map in advance. At each step it "senses" nearby cells, updates its internal model, and decides the next move using only what it currently knows. If it hits a dead end, it backtracks.

```
    """
```

```
    internal_model = {} # known_pos -> True(free) / False(obstacle)
```

```
    path_taken = [start]
```

```
    stack = [start]
```

```
    visited = set()
```

```
def sense(pos):
```

```
    """Reveal the true status of cells within sensor_range of pos."""
```

```
    r, c = pos
```

```
for dr in range(-sensor_range, sensor_range + 1):
    for dc in range(-sensor_range, sensor_range + 1):
        neighbor = (r + dr, c + dc)
        if 0 <= neighbor[0] < true_env.rows and 0 <= neighbor[1] < true_env.cols:
            internal_model[neighbor] = true_env.is_free(neighbor)
```

```
def neighbors(pos):
    r, c = pos
    candidates = [(r - 1, c), (r + 1, c), (r, c - 1), (r, c + 1)]
    return [p for p in candidates if internal_model.get(p, False)]
```

```
current = start
sense(current)
visited.add(current)
```

```
while current != goal:
    sense(current)
    options = [n for n in neighbors(current) if n not in visited]
```

```
if options:
    next_pos = options[0]
    stack.append(next_pos)
    visited.add(next_pos)
    path_taken.append(next_pos)
    current = next_pos
else:
```

```

    # Dead end -> backtrack (this is what makes it "online": it
    # learns about the obstacle only after arriving nearby)
    if len(stack) <= 1:
        return None, path_taken # no path found
    stack.pop()
    current = stack[-1]
    path_taken.append(current)

return path_taken, path_taken

```

```

def demo_scenario_3():
    print("\n=== SCENARIO 3: Mars Rover Online Search Agent ===")
    # '.' = safe, '#' = obstacle (unknown to rover until nearby), R start, G goal
    grid = [
        list(".....#"),
        list("###.#"),
        list("....."),
        list("###.."),
        list("....."),
    ]
    env = MarsGrid(grid)
    start, goal = (0, 0), (4, 5)

    path, full_trace = online_dfs_agent(env, start, goal)
    print(f"Final path to goal: {path}")

```

```

print(f"Total moves (incl. backtracking): {len(full_trace)}")

print("-> The rover only 'discovers' obstacles as it senses nearby cells,")

print(" updating its internal model and re-planning in real time.")

```

```

# =====

# SCENARIO 4: University Exam Timetabling (CSP)

# Backtracking Search + Forward Checking

# =====

```

```

def solve_exam_timetable(exams, time_slots, halls, conflicts, verbose=True):

```

```

    """

```

```

    exams:    list of exam names, e.g. ["Math", "Physics", "Chemistry", "English"]

```

```

    time_slots: list of available slots, e.g. ["Morning", "Afternoon", "Evening"]

```

```

    halls:    list of available halls, e.g. ["Hall A", "Hall B"]

```

```

    conflicts: dict exam -> set of exams that CANNOT share the same time slot

```

```

               (e.g. because some students are enrolled in both)

```

```

    Returns a dict {exam: (slot, hall)} satisfying all constraints, or None.

```

```

    Uses Backtracking Search with Forward Checking.

```

```

    """

```

```

    domains = {exam: [(s, h) for s in time_slots for h in halls] for exam in exams}

```

```

    assignment = {}

```

```

def is_consistent(exam, value):

```

```

    slot, hall = value

```

```

for other, (o_slot, o_hall) in assignment.items():
    if o_slot == slot and o_hall == hall:
        return False # one hall cannot host two exams simultaneously
    if other in conflicts.get(exam, set()) and o_slot == slot:
        return False # conflicting exams cannot share a time slot
return True

```

```

def forward_check(exam, value, local_domains):
    """Remove now-invalid options from domains of unassigned, conflicting exams."""
    slot, hall = value
    removed = {}
    for other in exams:
        if other in assignment or other == exam:
            continue
        if other in conflicts.get(exam, set()):
            pruned = [v for v in local_domains[other] if v[0] == slot]
            if pruned:
                removed[other] = pruned
                local_domains[other] = [v for v in local_domains[other] if v not in pruned]
            if not local_domains[other]:
                return None # domain wiped out -> failure, must backtrack
    return removed

```

```

def restore(local_domains, removed):
    for exam, values in removed.items():
        local_domains[exam].extend(values)

```



```

def backtrack(local_domains):
    if len(assignment) == len(exams):
        return dict(assignment)

    unassigned = [e for e in exams if e not in assignment]
    exam = unassigned[0] # (could use MRV heuristic: pick smallest domain)

    for value in list(local_domains[exam]):
        if is_consistent(exam, value):
            assignment[exam] = value
            removed = forward_check(exam, value, local_domains)

            if verbose:
                print(f" Assign {exam} -> {value} (forward checking OK: {removed is not None})")

            if removed is not None:
                result = backtrack(local_domains)
                if result:
                    return result

            # undo
            restore(local_domains, removed or {})
            del assignment[exam]

    return None

```

```
return backtrack(domains)
```

```
def demo_scenario_4():
```

```
    print("\n=== SCENARIO 4: Exam Timetabling (CSP: Backtracking + Forward Checking)
    ===")
```

```
    exams = ["Math", "Physics", "Chemistry", "English"]
```

```
    time_slots = ["Morning", "Afternoon", "Evening"]
```

```
    halls = ["Hall A", "Hall B"]
```

```
# Students taking Math also take Physics; Chemistry conflicts with English
```

```
conflicts = {
```

```
    "Math": {"Physics"},
```

```
    "Physics": {"Math"},
```

```
    "Chemistry": {"English"},
```

```
    "English": {"Chemistry"},
```

```
}
```

```
result = solve_exam_timetable(exams, time_slots, halls, conflicts)
```

```
print("\nFinal Timetable:")
```

```
if result:
```

```
    for exam, (slot, hall) in result.items():
```

```
        print(f" {exam:10s} -> {slot:9s} | {hall}")
```

```
else:
```

```
    print(" No valid timetable found.")
```

```
# =====  
# SCENARIO 5: Strategic Game AI - Minimax & Alpha-Beta Pruning  
# =====
```

```
class GameTreeNode:
```

```
    """A node in a game tree. Leaf nodes carry a numeric evaluation value."""
```

```
    def __init__(self, value=None, children=None):
```

```
        self.value = value
```

```
        self.children = children or []
```

```
    def is_leaf(self):
```

```
        return not self.children
```

```
def minimax(node, depth, maximizing_player):
```

```
    """
```

```
    Classic Minimax: MAX picks the highest value, MIN picks the lowest.
```

```
    Returns the minimax value of `node` .
```

```
    """
```

```
    if node.is_leaf() or depth == 0:
```

```
        return node.value
```

```
    if maximizing_player:
```

```

    best = -math.inf

    for child in node.children:

        best = max(best, minimax(child, depth - 1, False))

    return best

else:

    best = math.inf

    for child in node.children:

        best = min(best, minimax(child, depth - 1, True))

    return best

```

```

def minimax_alpha_beta(node, depth, alpha, beta, maximizing_player,
nodes_evaluated=None):

```

```

    """

```

```

    Minimax with Alpha-Beta Pruning.

```

```

    alpha = best value MAX can guarantee so far

```

```

    beta = best value MIN can guarantee so far

```

```

    Prunes remaining branches once alpha >= beta.

```

```

    `nodes_evaluated` (optional list with one int) tracks how many leaves were visited,

```

```

    to demonstrate the efficiency gain vs plain Minimax.

```

```

    """

```

```

    if nodes_evaluated is not None:

```

```

        nodes_evaluated[0] += 1

```

```

    if node.is_leaf() or depth == 0:

```

```

        return node.value

```

```

if maximizing_player:
    value = -math.inf

    for child in node.children:
        value = max(value, minimax_alpha_beta(child, depth - 1, alpha, beta, False,
nodes_evaluated))

    alpha = max(alpha, value)

    if alpha >= beta:
        break # beta cutoff -> prune remaining siblings

    return value

else:
    value = math.inf

    for child in node.children:
        value = min(value, minimax_alpha_beta(child, depth - 1, alpha, beta, True,
nodes_evaluated))

    beta = min(beta, value)

    if alpha >= beta:
        break # alpha cutoff -> prune remaining siblings

    return value

```

```

def evaluation_function(units, resources, territory, enemy_strength):
    """
    Score = 5*Units + 4*Resources + 3*Territory - 2*EnemyStrength
    (as specified in the document's Scenario 5, part iv)
    """

    return 5 * units + 4 * resources + 3 * territory - 2 * enemy_strength

```

```

def demo_scenario_5():

    print("\n=== SCENARIO 5: Minimax & Alpha-Beta Pruning ===")

    # Recreate the document's example tree:

    #      MAX
    #     /  \
    #    MIN  MIN
    #   / \  / \
    #  3  5 2  9

    leaf_3, leaf_5, leaf_2, leaf_9 = (GameTreeNode(3), GameTreeNode(5),
                                       GameTreeNode(2), GameTreeNode(9))

    min_left = GameTreeNode(children=[leaf_3, leaf_5])
    min_right = GameTreeNode(children=[leaf_2, leaf_9])
    root = GameTreeNode(children=[min_left, min_right])

    plain_result = minimax(root, depth=3, maximizing_player=True)

    print(f"Minimax result: {plain_result} (matches document: MAX picks 3)")

    nodes_evaluated = [0]

    ab_result = minimax_alpha_beta(root, depth=3, alpha=-math.inf, beta=math.inf,
                                    maximizing_player=True, nodes_evaluated=nodes_evaluated)

    print(f"Alpha-Beta result: {ab_result} | leaf nodes evaluated: {nodes_evaluated[0]} (out of 4 total)")

    print("-> Alpha-Beta reaches the same optimal decision while pruning the 'X' branch (the 9).")

```

```
score = evaluation_function(units=10, resources=8, territory=5, enemy_strength=6)
print(f"\nExample evaluation function score: {score}")
print("Score = 5*Units + 4*Resources + 3*Territory - 2*EnemyStrength")
```

```
# =====
# MAIN
# =====
```

```
if __name__ == "__main__":
    demo_scenario_1()
    demo_scenario_2()
    demo_scenario_3()
    demo_scenario_4()
    demo_scenario_5()
```